

GRADES 7-10



EMERGENCY SUB PLANS PACK

for **DIGITAL TECHNOLOGIES**

★ Keep learning on track—even when plans change. ★

VOLUME 3

**COMPUTER
SYSTEMS**



CPU



RAM



STORAGE


**READY
WHEN YOU
NEED IT!**

TASK CARD

12

Write a program that
flashes an LED on
for 2 seconds and
off for 2 seconds.



NETWORKS

SHORT ANSWER QUESTIONS



1. What is the main purpose of a router?

2. What is an IP address?

3. Give one example of a wireless network.

BINARY BASICS

Convert the following
binary numbers to decimal.

1010	=	_____
1101	=	_____
1111	=	_____
10011	=	_____



**NO-PREP
ACTIVITIES**



**READY-TO-USE
RESOURCES**



**PERFECT FOR
EMERGENCY
LESSONS**



**GREAT FOR
SUBSTITUTE
TEACHERS**



**ENGAGING
DIGITAL TECH
TASKS**



**PRINT OR
DIGITAL USE**



YOUR **LIFESAVER** IN THE CLASSROOM

 SAVE TIME

 REDUCE STRESS

 KEEP STUDENTS
ON TASK



**MADE BY TEACHERS
FOR TEACHERS**

Set this up while you are well

15 MINUTES

You are reading this because you are **well**. Spend fifteen minutes now and this pack will run itself on a day when you are not.

The one job you have today

Print pages 3–6 of this pack and leave them in a clearly labeled folder where your substitute teacher will find them — top drawer, on the desk, or with the front office. Everything a substitute needs is in those four pages.

You do not need to print the lessons in advance. The substitute teacher chooses one on the day and photocopies it.

SETTING IT UP • FIFTEEN MINUTES, ONCE

- 1 **Print the whole pack once.** Put it in a display folder or bind it. Write your name and room number on the cover.
- 2 **Print pages 3–6 a second time** and clip them to the front. These are the pages a substitute teacher reads first.
- 3 **Tell your department head where it lives.** On a genuine emergency day you will not be answering your phone, so somebody else needs to know.
- 4 **Cross off the lessons you have already used.** There is a tracker on page 4. Two years of substitute days will not repeat if you keep it current.
- 5 **Optional: leave a class list** and a note about any students who need specific support, seating or a reader.

Why these lessons are unplugged

On a substitute day the laptop cart is often locked, the login server is down, or nobody knows the room password. Every lesson here runs on paper.

Each lesson also has an **If the computers are working** note, so the same sheet still works when technology cooperates.

Why they are not busy work

Every lesson targets a genuine Digital Technologies concept and produces marked evidence you can put in a folder.

If you are away for a week, running four of these in sequence is a defensible unit of work, not a holding pattern.

A word about the answer keys

Every answer key sits on the **last page of its lesson**. If you hand a lesson to a substitute teacher, hand them the answer key too — it is the single thing that lets a non-specialist mark with confidence and answer a student who says “*but why?*”

This is Volume Three

Volume 3 stands completely on its own and shares nothing with Volumes 1 or 2. There is no binary conversion, no flowchart, no bubble sort, no data cleaning and no compression here. Owning all three gives you thirty substitute lessons that never repeat.

You do not need to know this subject

You have walked into a Digital Technologies class. You may have never taught the subject. That is completely fine — this pack was written for you.

Read this and you are ready

You do not need to know how to code. Every lesson is a paper worksheet with a full answer key. Your job is to hand it out, keep the room settled, and check work as students finish. You are not expected to teach the content.

DO THESE FOUR THINGS

- 1 **Pick one lesson** from the table on the next page. Any of them works for any class from Grade 7 to Grade 10. If you cannot decide, use Lesson 03.
- 2 **Photocopy the student pages only.** Each lesson tells you exactly which page numbers to copy. One copy per student.
- 3 **Keep the teacher page and the answer key.** Do not photocopy those.
- 4 **Read the teacher page once** before the bell. It takes about two minutes and includes the exact words you can say to start the lesson.

If a student says “we have done this”

Ask them to prove it by completing the extension section at the end. If they genuinely have, switch them to a different lesson from the table — the pack has ten, and they will not have done them all.

If you finish early

Every lesson has an extension task on its final student page. Point students there. There is also a discussion prompt on the teacher page you can run as a whole class for the last five minutes.

What to leave for the regular teacher

Collect the completed sheets. Fill in the handover note on the last page of this pack — it takes ninety seconds and tells the regular teacher which lesson you used, how far the class got, and anything that happened.

Leave both on the desk.

Every lesson is standalone. There is no order and no prerequisite — pick whichever suits the class in front of you.

#	LESSON	WHAT IT COVERS	DIFFICULTY	MINS	STUDENT PAGES
01	Lists, Indexes and Off-by-One	Data structures and indexing	★★	55	7-9
02	True, False and In Between	Boolean logic and conditions	★★	55	12-14
03	Inside the Box	Hardware and how a CPU works	★	55	17-19
04	Where Did the File Go?	File systems, paths and backups	★	55	22-24
05	Is It Finished?	Requirements, evidence and evaluation	★★	60	27-29
06	Teaching a Machine	Machine learning, training data and bias	★★	60	32-34
07	The Internet's Address Book	DNS, addresses and protocols	★★	55	37-39
08	How a Search Engine Ranks	Indexing, ranking and relevance	★★★	60	42-44
09	Machines That Sense and Act	Control algorithms and thresholds	★★	55	47-49
10	Whose Work Is It?	Copyright, licensing and attribution	★	55	52-54

Easiest to supervise

03, 04 and 10. Reading, judgment and argument, with a definite answer waiting at the end of each one.

Best for a quiet, working class

01, 02 and 08. Puzzle-like with exact answers. Strong students will race; that is what the extensions are for.

Best for a restless class

06, 07 and 09. Counting patterns, taking an address apart and steering a robot through a maze. Noisier, but productively so.

Used-lesson tracker · for the regular teacher

LESSON	DATE USED	CLASS	NOTES
01 — Lists and indexes			
02 — Boolean logic			
03 — Hardware			
04 — Files and backups			
05 — Requirements			
06 — Machine learning			
07 — DNS and addresses			
08 — Search ranking			
09 — Robots and control			
10 — Copyright and licenses			

The first ten minutes

SCRIPT

The first four minutes decide the next fifty. Here is a script that works even if you have never met this class.

SAY THIS AT THE DOOR

"Come in, sit down, and leave your bags off the desks. You will not need a computer today. Everything is on paper."

THEN, ONCE THEY ARE SEATED

"Your teacher is away. My name is _____. We are doing a proper Digital Technologies lesson today, not a free period. There is a worksheet, it gets collected at the end, and your teacher will see it."

Why that last sentence matters

The single most useful thing you can say is that the work will be seen by their regular teacher. It is also true — there is a handover note at the back of this pack.

THE SHAPE OF EVERY LESSON IN THIS PACK

TIME	WHAT HAPPENS	WHAT YOU DO
0-5	Settle, hand out sheets	Use the script above
5-10	Read the opening page together	Ask one student to read it aloud
10-45	Students work through the tasks	Circulate. Check answers against the key
45-55	Extension task for those who finish	Point them to the last student page
55-60	Collect sheets, pack up	Fill in the handover note

If a student refuses to work

Offer the smaller ask: *"Just do question one and show me."* Most refusals are about not knowing where to start, not defiance.

Note it on the handover sheet. Do not escalate a single lesson into a battle you will not be there to finish.

If the whole class claims they cannot do it

Do the first question together on the board. Every lesson has a worked example on the first student page.

If it is genuinely too hard for the grade level, switch to Lesson 03 or Lesson 10 — both need no prior knowledge at all.

Photocopying reminder

Student pages only. Each lesson's teacher page states the exact page range. The **answer key is always the last page** of the lesson — keep it.

Lists, Indexes and Off-by-One

2 MINUTES

Students look up items in a list by index, trace four operations that change it, then hunt down the single most common bug in programming: the loop that runs one time too many or too few.

You do not need to know this subject

Every answer is one word or one short list, and every list is written out after each step in the key. There is no code to write.

Photocopy these pages only

Pages 7-9

One per student. **Do not photocopy this page or the answer key.**

What students need

A pen. Crossing out is better than rubbing out, so the working stays visible.

WHAT TO SAY TO START

"A list is a row of numbered boxes, and the numbering starts at zero, not one. Almost every mistake in this lesson comes from that one sentence, and so does almost every mistake made by professionals."

HOW THE LESSON RUNS

TIME	WHAT HAPPENS	WHAT YOU DO
0-5	Hand out sheets, settle	Use the door script on page 5
5-12	Read the panel and the worked example	Say the word index out loud repeatedly
12-24	Task 1.1 — look things up	Every answer is definite. Grade as you circulate
24-36	Task 1.2 — trace the operations	Key has the list after every step
36-48	Task 1.3 — the off-by-one hunt	Four bugs, four fixes
48-55	Tasks 1.4 and 1.5, collect sheets	Discussion prompt below

Questions students will ask

"Why does it start at zero?"

The index is really a distance from the start of the list. The first item is zero steps away. Task 1.5 asks them to work that out.

"What does the minus one mean?"

Count backwards from the end. [-1] is the last item, whatever the length.

"Is index 8 of an 8-item list really an error?"

Yes. The valid indexes are 0 to 7. This trips up everybody once and then never again.

If they finish early

Task 1.4 moves to a two-dimensional list, which is how every seating plan, spreadsheet and game board is stored. Strong students can then work out how many indexes a chessboard needs and write the one for the bottom-left square.

If the computers are working

Students can type a list into any Python environment and try the index that should fail. The error message names the exact problem, which is unusually helpful for a computer.

Curriculum focus · for the regular teacher's records

Data structures: lists and arrays; indexing and iteration; identifying and correcting logic errors in loops. Discussion prompt for the last five minutes: *"Counting from zero causes thousands of bugs a year. So why has no language ever fixed it?"*

Numbered boxes

14 MIN

NAME

CLASS

DATE

A list is a row of boxes with numbers on them

The number is the **index**. The first box is index **0**, not 1. A list of five items has indexes 0, 1, 2, 3 and 4 — and asking for index 5 is an error, not an empty box.

worked.py

PYTHON

```
names = ["Ari", "Bec", "Cam", "Dev", "Eli"]

print(names[0]) # Ari
print(names[3]) # Dev
print(len(names)) # 5
print(names[-1]) # Eli
print(names[5]) # error
```

What to notice

len gives the number of items, so the last valid index is always **len - 1**.

[-1] counts backwards from the end and always finds the last item.

The final line does not print a blank. It stops the program.

Task 1.1 · Look it up

DO

subjects.py

PYTHON

```
subjects = ["Math", "English", "Science", "Art",
            "PE", "Music", "History", "Digital Tech"]
```

#	QUESTION	ANSWER
a	subjects[0]	
b	subjects[4]	
c	len(subjects)	
d	The index of "Music"	
e	subjects[-1]	
f	subjects[8]	
g	The highest index that works	

h. Write the rule connecting **len** to the highest valid index: _____

Task 1.1 · looking things up

a. Math · **b.** PE · **c.** 8 · **d.** 5 · **e.** Digital Tech · **f.** An error — the program stops. `IndexError: list index out of range` · **g.** 7.

h. The highest valid index is always `len - 1`. Any student who writes that sentence has the whole lesson.

Watch for students answering **f** with ‘nothing’ or ‘blank’. The list does not contain an empty box at index 8; there is no box at all.

Task 1.2 · the trace

STEP	THE LIST	LENGTH
Start	[4, 9, 2]	3
1	[4, 9, 2, 7]	4
2	[4, 5, 9, 2, 7]	5
3	[5, 9, 2, 7]	4
4	[5, 9, 8, 7]	4
5	[5, 9, 8, 7, 5]	5

a. Steps 2 and 3 both did — inserting and removing shift everything after them.

b. A loop that has already decided it will run six times does not notice that item 4 is now item 3. Removing items from a list while looping over it skips items, and it is one of the most confusing bugs a beginner meets.

Task 1.3 · the four off-by-one bugs

#	WHAT GOES WRONG	THE FIX
a	Runs for indexes 0–4 only. The sixth mark is silently ignored	<code>range(6)</code> or <code>range(len(scores))</code>
b	Starts at index 1, so the first mark is skipped	<code>range(0, 6)</code> , or simply <code>range(6)</code>
c	Reaches index 6, which does not exist. Crashes on the last pass	<code>range(6)</code>
d	Same fault: <code>i</code> reaches 6 and the lookup fails	<code>while i < 6:</code>

a and b are the dangerous ones. c and d crash, which tells you immediately that something is wrong. a and b run perfectly and quietly produce an average of the wrong five scores — a number nobody will ever question.

`range(len(scores))` is the answer worth teaching: it is correct whatever the list length becomes later.

Task 1.4 · the seating plan

a. Gus · **b.** `seats[2][2]` · **c.** 12 · **d.** There are only rows 0, 1 and 2. Row 3 does not exist.

e. `seats[7][5]`. Almost every class writes `[8][6]` first — the same off-by-one, arriving in a new costume.

The first index is the row, the second is the seat. Reading it as ‘the third item of the second row’ is the habit worth building.

Task 1.5 · the closing question

The list is one continuous block of memory. The index tells the machine how far to move from the start to reach the item you want — and the first item requires no movement at all, so it is zero.

Accept any answer reaching ‘it counts the steps from the beginning, not the items’.

Worth adding: languages that start at one have to subtract one every single time an item is fetched. The bug never disappears; it just moves from the programmer into the machine.

Finding the rule in the examples

16 MIN

NAME

CLASS

DATE

Two ways to make a machine decide

Written rules: a person decides what counts and types it in. You can read it, argue with it, and fix it.

Learned rules: the machine is shown examples already labeled with the right answer, and it finds whatever pattern separates them. Nobody chooses the pattern, and often nobody can say afterwards what it was.

Task 6.1 · Train the filter

ANALYZE

Eight emails have already been labeled by a human. Count how often each feature appears in each group.

EMAIL	LABEL	SHOUTING SUBJECT	2 OR MORE LINKS	UNKNOWN SENDER	SAYS 'FREE'
1	Spam	Y	Y	Y	Y
2	Spam	Y	Y	Y	N
3	Spam	N	Y	Y	Y
4	Spam	Y	N	N	Y
5	Not spam	N	N	N	N
6	Not spam	N	N	N	N
7	Not spam	N	Y	Y	N
8	Not spam	N	N	N	Y

COUNT	SHOUTING	2+ LINKS	UNKNOWN SENDER	'FREE'
How many of the 4 spam emails				
How many of the 4 real emails				

- Which single feature separates the two groups best? _____
- Score one point per feature present. What is the lowest score of any spam email? _____. The highest score of any real email? _____
- Write your rule: mark as spam if the score is at least _____

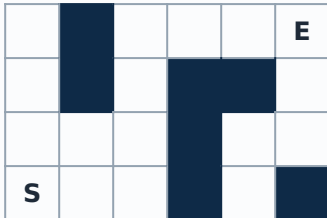
Finding a way through

18 MIN

Task 9.2 · The maze

PLAN

The robot starts at **S** facing right and must reach **E**. Shaded squares are walls. It can only move forward one square at a time or turn 90 degrees on the spot.



Write the instructions

Use FORWARD n , TURN LEFT and TURN RIGHT .

a. Forward moves in total: _____

b. Turns in total: _____

c. Now the robot has **no map** — only a sensor that tells it whether there is a wall on its right. Write a rule, repeated forever, that would still get it out of a maze like this one.

Task 9.3 · Choosing the threshold

CALCULATE

A light sensor reports a number from 0 to 1023. In the lit room it reads about **800**. With the lights off it reads about **120**. The lamp should switch on when it is dark.

#	QUESTION	ANSWER
a	Choose a threshold and write the rule	
b	Why is a threshold of 790 a bad choice?	
c	At dusk the reading drifts around your threshold. What does the lamp do?	
d	Fix it using two numbers instead of one	

Photocopy this page or fill it in directly. Ninety seconds of your time saves the regular teacher a great deal of guesswork.

SUBSTITUTE TEACHER

CLASS

DATE

PERIOD

Lesson used

CHECK ONE

☐ 01 · Lists, Indexes and Off-by-One☐ 06 · Teaching a Machine☐ 02 · True, False and In Between☐ 07 · The Internet's Address Book☐ 03 · Inside the Box☐ 08 · How a Search Engine Ranks☐ 04 · Where Did the File Go?☐ 09 · Machines That Sense and Act☐ 05 · Is It Finished?☐ 10 · Whose Work Is It?**How far did the class get?**

- ☐ Everyone finished the main tasks
- ☐ Most finished; some are partway
- ☐ We got through about half
- ☐ We did not get far — see notes

Sheets

- ☐ Collected and left on the desk
- ☐ Students kept them
- ☐ Graded against the answer key

Anything the regular teacher should know

NOTES

Students to mention · for any reason, good or otherwise

NAMES

Thank you

Walking into an unfamiliar subject is not easy. Leaving a clear note behind is the part that makes the next teacher's day work, and it is genuinely appreciated.

Terms of use

Licensed for **one teacher in one classroom**. You may print and photocopy these pages for the students you personally teach, and you may pass the pack to a substitute teacher covering your own classes.

You may not redistribute it to colleagues, upload it to a school intranet or shared drive, resell it, or post it publicly. Additional licenses for a faculty or whole school are available at a discount.

© Digital Vector · digitalvector.com.au

Curriculum alignment

CSTA K-12 Computer Science Standards, levels 2 and 3A. Across the ten lessons this volume touches:

- Data structures: lists, indexing and iteration
- Boolean logic, truth tables and branching conditions
- Hardware components and the fetch-execute cycle
- File management, backup and secure deletion
- Requirements, design criteria and evaluation with evidence
- Machine learning, training data and bias
- Network addressing, DNS and secure transmission
- Searching, indexing and ranking algorithms
- Control systems, sensors, thresholds and safety
- Intellectual property, licensing and attribution

Individual lesson teacher pages name the specific focus for that lesson.

The rest of the series**Volume 1 · Foundations**

Algorithms, binary, trace tables, flowcharts, cyber attacks, networks, sorting, debugging, AI and digital footprints.

Volume 2 · Data, Systems and Design

Data cleaning, misleading charts, databases, input-process-output, compression, encryption, interface design, functions, project planning and the cost of free apps.

Volume 4 · Media, Evidence and Impact

Images and sound as data, stacks and queues, digital forensics, version control, network design, accessibility, algorithmic decisions, IoT security and automation.

digitalvector.com.au

Physical Computing with the Raspberry Pi 5 · AI Literacy for Secondary Students · and more.

Found an error, or want a lesson on something else?

Feedback is genuinely read. If a lesson did not land with your class, or you need a sub plan on a topic that is not here, get in touch through digitalvector.com.au and it will be considered for the next update.

EMERGENCY SUB PLANS PACK

Digital Technologies · Grades 7-10

Volume Three · ten lessons · no preparation · no computers required